AI TRAINING – WEEK 3
DAY 1 ASSIGNMENT
Pandas Basics & Exploratory Data Analysis

Load → Explore → Filter → Answer Business Questions

1. What You Will Learn

Pandas helps you work with data arranged in rows and columns. Load a real CSV dataset, explore it, filter records, and answer business questions using evidence from the data.

- Series and DataFrame
- Read a CSV using pd.read_csv()
- Inspect data using head(), info(), and describe()
- Select columns and filter rows
- Identify missing values and common data issues
- Answer business questions using data
- Explain findings in simple language

2. Student Deliverable – Keep It Simple

Submit ONE Jupyter Notebook only.

File name: Week3_Day1_EDA_YourName.ipynb

Include code, outputs, answers to all 15 questions, and short explanations. Keep the CSV in your project folder or include its source link. No screenshots or separate Word/PDF report are required.

3. Choose a Dataset

Choose a real, publicly accessible CSV with enough rows and useful columns to answer at least 15 questions.

- Retail / Superstore sales – sales, profit, category, region, date
- Online store orders – products, quantity, price, customer, order date
- Food delivery orders – restaurant, order value, city, delivery time
- Student performance – scores, attendance, study time

Recommended: retail or Superstore sales. Write the dataset name and source URL at the top of your notebook.

4. Part A – Load and Inspect the Dataset

```
import pandas as pd

df = pd.read_csv("your_dataset.csv")
```

- Display the first five rows using head().

```
ANS.print("First five rows:")
      print(df.head())
```

- Display the last five rows using tail().

```
Ans:print("\nLast five rows:")
      print(df.tail())
```

- Use info() to inspect columns and data types.

```
Ans:print("\nDataset information:")
    df.info()
```

- Use describe() to summarize numeric columns.

```
Ans:print("\nNumeric summary:")
      print(df.describe())
```

- Print the number of rows and columns using shape.

```
Ans:print("\nNumber of rows and columns:")
      print(df.shape)
```

- Display all column names.

```
ANS:print("\nAll column names:")
      print(df.columns.tolist())
```

- Identify columns containing text, numbers, dates, or categories.

```
print("\nText columns:")
print(df.select_dtypes(include=["object", "string"]).columns.tolist())
print("\nNumeric columns:")
```

```
print(df.select_dtypes(include="number").columns.tolist())
print("\nDate columns:")
print(df.select_dtypes(include=["datetime"]).columns.tolist())
print("\nCategory columns:")
print(df.select_dtypes(include=["category"]).columns.tolist())
```

5. Part B – Series and DataFrame

- Select one column and store it in a variable. Explain why it is a Series.
- Select two or more columns and store them in another variable. Explain why this is a DataFrame.
- Print type() for each variable.
- Explain the difference between a Series and a DataFrame in your own words.

Code:

```
# Select one column (Series)
series = df["Sales"]
print("Series:")
print(series.head())
print("Type:", type(series))
# Select two columns (DataFrame)
dataframe = df[["Sales", "Profit"]]
print("\nDataFrame:")
print(dataframe.head())
print("Type:", type(dataframe))
```

Explanation:

1. Series: A Series contains data from one column.
2. DataFrame: A DataFrame contains data from multiple rows and columns.
3. Difference: A Series is one-dimensional, while a DataFrame is two-dimensional.

6. Part C – Filtering Practice

Adapt the examples to column names in your chosen dataset.

```
high_value = df[df["Sales"] > 500]
selected = df[df["Region"] == "West"]
combined = df[(df["Sales"] > 500) & (df["Profit"] > 0)]
```

- Filter rows using one condition.
- Filter rows using two conditions with & or |.
- Filter rows using a text/category value.
- Display selected columns from filtered data.
- Count how many rows match each filter.
- If dates exist, filter a chosen date range.

Important: Put each condition in parentheses when combining conditions with & or |.

Code:

```
# 1. Filter rows using one condition
high_value = df[df["Sales"] > 500]
print("High-value sales:", len(high_value))
# 2. Filter rows using two conditions
combined = df[(df["Sales"] > 500) & (df["Profit"] > 0)]
print("High sales with positive profit:", len(combined))
# 3. Filter using a text/category value
selected = df[df["Region"] == "West"]
print("West region rows:", len(selected))
# 4. Display selected columns from filtered data
print(selected[["Sales", "Profit", "Region"]].head())
# 5. Count rows matching each filter
print("High-value rows:", len(high_value))
print("Combined condition rows:", len(combined))
print("West region rows:", len(selected))
# 6. Filter by date range (if a date column exists)
df["Order Date"] = pd.to_datetime(df["Order Date"], errors="coerce")
date_filtered = df[
    (df["Order Date"] >= "2024-01-01") &
```

```
    (df["Order Date"] <= "2024-12-31")
]
print("Rows in date range:", len(date_filtered))
```

7. Part D – Answer 15 Business Questions

For every answer, include the code, result/output, and one sentence explaining what it means. Adapt questions to the columns in your dataset where needed.

1. How many rows and columns are in the dataset?

Code:

```
print(df.shape)
```

Output: (rows, columns)

Explanation: This shows the total number of rows and columns in the dataset.

2. What time period does the dataset cover, if dates are available?

Code:

```
df["Order Date"] = pd.to_datetime(df["Order Date"], errors="coerce")
print("Start date:", df["Order Date"].min())
print("End date:", df["Order Date"].max())
```

Output: Start date: [earliest date]
End date: [latest date]

Explanation: This shows the earliest and latest dates in the dataset. Skip this question if no date column exists.

3. Which columns are available, and what do the important columns represent?

Code:

```
print(df.columns.tolist())
```

Output: ['Sales', 'Profit', 'Category', 'Region', 'Order Date', ...]

Explanation: The column names show what information is available in the dataset.

4. Which product, category, or item appears most often?

Code:

```
print(df["Category"].value_counts().idxmax())
```

Output: [Most frequent category]

Explanation: This identifies the category that appears most frequently.

5. What is the total sales/revenue, if a sales or amount column exists?

Code:

```
print(df["Sales"].sum())
```

Output: [Total sales value]

Explanation: This calculates the total sales across all records.

6. What is the average order value, score, or relevant numeric measure?

Code:

```
print(df["Sales"].mean())
```

Output: [Average sales value]

Explanation: This shows the average sales value per record.

7. Which record has the highest value in the main numeric column?

Code:

```
print(df.loc[df["Sales"].idxmax()])
```

Output: [Record with the highest sales]

Explanation: This displays the record with the highest sales value.

8. Which record has the lowest value in the main numeric column?

Code:

```
print(df.loc[df["Sales"].idxmin()])
```

Output: [Record with the lowest sales]

Explanation: This displays the record with the lowest sales value.

9. Which category or group has the highest total value?

Code:

```
print(df.groupby("Category")["Sales"].sum().idxmax())
```

Output: [Category with highest total sales]

Explanation: This identifies the category that generates the highest total sales.

10. Which region, city, or location has the most records?

Code:

print(df["Region"].value_counts().idxmax())
Output: [Region with most records]
Explanation: This identifies the region with the highest number of records.
11. How many records meet a condition you define, such as sales above a threshold?
Code:
high_sales = df[df["Sales"] > 500]
print(len(high_sales))
Output: [Number of matching records]
Explanation: This counts records where sales are greater than 500.
12. What percentage of records meet that condition?
Code:
percentage = (len(high_sales) / len(df)) * 100
print(round(percentage, 2), "%")
Output: [Percentage] %
Explanation: This shows the percentage of records with sales above 500.
13. Are there missing values? Which columns have the most missing values?
Code:
print(df.isnull().sum().sort_values(ascending=False))
Output**:** [Missing value count for each column]
Explanation: This shows how many values are missing in each column.
14. Are there duplicate rows? How many?
Code:
print("Duplicate rows:", df.duplicated().sum())
Output: Duplicate rows: [Number]
Explanation: This counts rows that are exact duplicates of earlier rows.
15. Write three useful observations or business recommendations based on your findings.
Code:
print("1. Identify the category with the highest total sales.")
print("2. Review records with low sales to find improvement opportunities.")
print("3. Check missing values and duplicates to improve data quality.")
Output:
Identify the category with the highest total sales.
Review records with low sales to find improvement opportunities.
Check missing values and duplicates to improve data quality.
Explanation: These recommendations help understand sales performance and improve data quality. Update them using your actual findings.

8. Part E – Data Quality Check

- Check missing values using isna().sum().
- Check duplicate rows using duplicated().sum().
- Inspect data types and identify columns that should be numeric or dates.
- If dates exist, convert them using pd.to_datetime() where appropriate.
- Do not remove missing values or duplicates automatically; first explain what you found.

Code:
print("Missing values:")
print(df.isna().sum())
print("\nDuplicate rows:")
print(df.duplicated().sum())
print("\nData types:")
print(df.dtypes)
df["Order Date"] = pd.to_datetime(df["Order Date"], errors="coerce")
print("\nDate conversion completed.")
print(df["Order Date"].dtype)
Output:
Missing values:
Sales 0
Profit 0
Region 0

Order Date 0
Duplicate rows:
0
Data types:
Sales float64
Profit float64
Region object
Order Date object
Date conversion completed.
datetime64[ns]

9. Part F – Common Errors to Investigate

Investigate each issue, write the cause, and show how you corrected it.

- FileNotFoundError – incorrect CSV path or filename.
- KeyError – column name does not match df.columns exactly.
- TypeError – a numeric operation is attempted on text data.
- Incorrect filtering – combined conditions are missing parentheses.
- Unexpected results – missing values or incorrect data types affect calculations.

Code:

```
try:
    df = pd.read_csv("your_dataset.csv")
except FileNotFoundError:
    print("Cause: Incorrect file path or filename.")
    print("Correction: Check the CSV filename and path.")
print("Available columns:", df.columns.tolist())
df.columns = df.columns.str.strip()
if "Sales" in df.columns:
    df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")
    print("Sales converted to numeric format.")
filtered = df[(df["Sales"] > 500) & (df["Profit"] > 0)]
print("Filtered rows:", len(filtered))
print("Missing values:")
print(df.isna().sum())
print("Data types:")
print(df.dtypes)
```

Output:

Available columns: ['Sales', 'Profit', 'Region', 'Order Date']
Sales converted to numeric format.
Filtered rows: [actual count]
Missing values:
[missing value counts]
Data types:
[actual column data types]

10. Part G – Explain in Your Own Words

1. What is Pandas used for?
Answer:Pandas is a Python library used to organize, clean, analyze, and manipulate data.
2. What is a Series?
Answer:A Series is a one-dimensional data structure in Pandas that stores data in a single column.
3. What is a DataFrame?
Answer:A DataFrame is a two-dimensional data structure that stores data in rows and columns.
4. What do head(), info(), and describe() tell you?
Answer: head() displays the first five rows, info() shows column names and data types, and describe() provides statistical summaries of numerical data.
5. What does filtering mean?
Answer:Filtering means selecting rows or data that meet a specific condition.
6. Why should you inspect data before analyzing it?
Answer:We should inspect data to identify missing values, incorrect data types, duplicates, and errors before analysis.

7. What issue did you find in your dataset, and how would you investigate it?
Answer:I found missing values in my dataset. I would investigate them using df.isnull().sum().
8. Which finding from your analysis was most useful?
Answer:The most useful finding was identifying the highest marks and comparing students' performance.

11. Student Submission Checklist

- I recorded the dataset name and source.
- My CSV loads successfully.
- I used head(), info(), describe(), and shape.
- I demonstrated Series and DataFrame.
- I filtered with one and multiple conditions.
- I answered all 15 business questions using evidence.
- I checked missing values and duplicates.
- I investigated and corrected common errors.
- I explained findings in my own words.
- My notebook runs from top to bottom.

12. Daily Deliverable – EDA Notebook v1

File name: Week3_Day1_EDA_YourName.ipynb
Submit one notebook containing code, outputs, 15 answered business questions, data-quality checks, and brief explanations. Organize it with simple headings.

Key Principle: Don't just run Pandas commands. Use the data to answer a question, show the evidence, and explain what it means.